Aplicações Industriais: Hashing e Resolução de Entidade

Lucas Nissenbaum e Thiago Ramos
5 de Maio de 2023

IMPA:

Centro de pesquisa e ensino de pós-graduação, fundado pelo governo federal em 1952.

Objetivos:

- Realizar pesquisa em matemática.
- Treinar pesquisadores e professores.
- Disseminar conhecimento matemático na sociedade.
- Integrar matemática na ciência e na indústria.

70 anos
impa
Instituto de Matemática
Pura e Aplicada

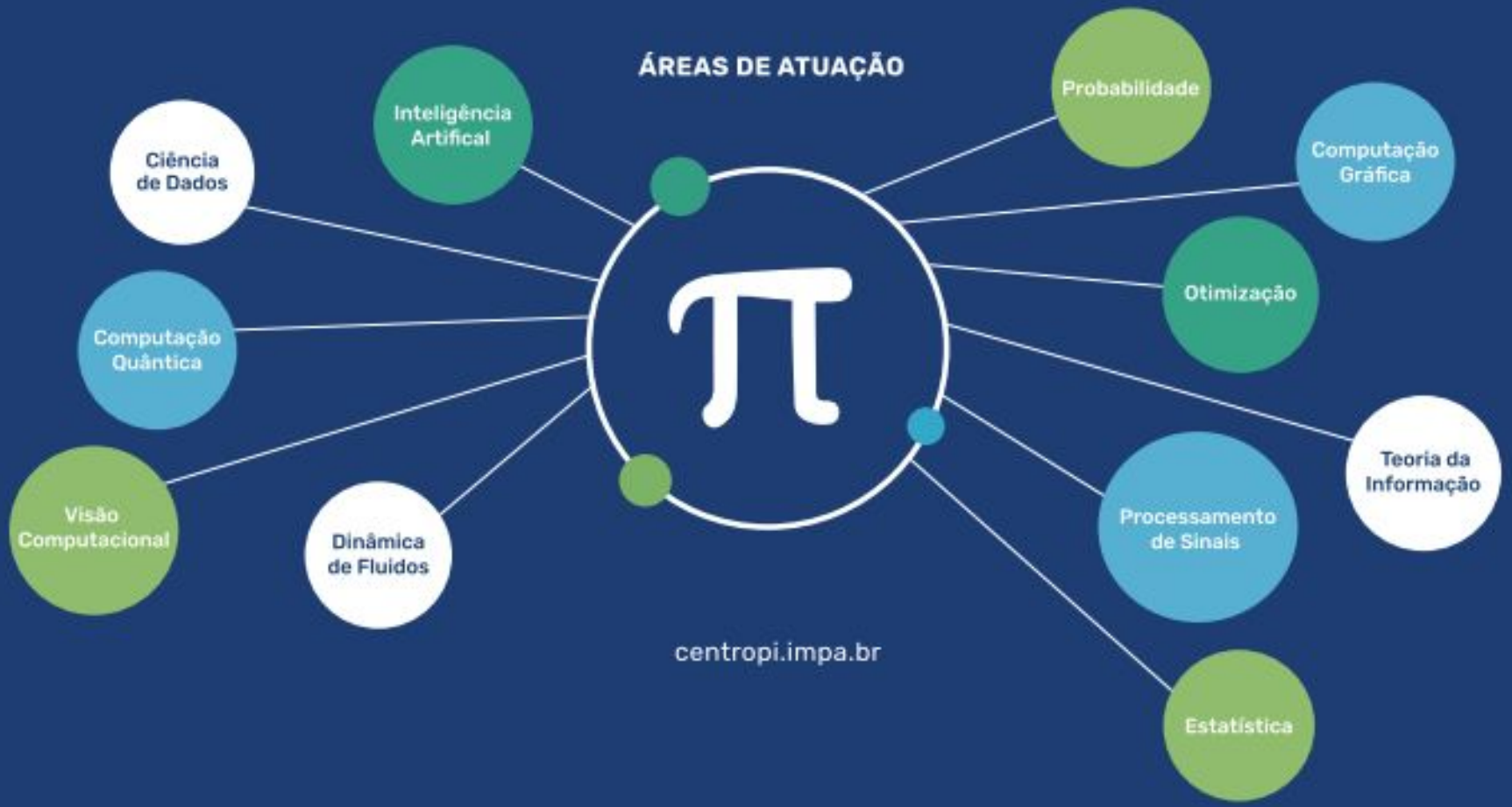
Centro de Inovação na Matemática Industrial

Centro Pi
Centro de Projetos e Inovação IMPA

Desenvolver projetos e soluções a problemas concretos que beneficiem de uma forte contribuição matemática.

Contribuir à transferência de tecnologia matemática e ao treinamento de profissionais para a indústria.

centropi.impa.br


ÁREAS DE ATUAÇÃO
Ciência de Dados
Inteligência Artifical
Probabilidade
Computação Gráfica
Computação Quântica
Otimização
Visão Computacional
Dinâmica de Fluidos
Teoria da Informação
Processamento de Sinais
Estatística
centropi.impa.br

dasa

Big Data

CARTESI

stone

VALE

McKinsey & Company

shape

equinor

rumo

carteira global

raízen
Energia que mobiliza

radix
Engenharia e Software

Centro Pi
Centro de Projetos e Inovação IMPA

Tribunal Superior Eleitoral

hurb

GPP
Grupo de Políticas Públicas
USP - ESALQ

PETROS

75 anos impa
Instituto de Matemática Pura e Aplicada

Projeto: DASA

- Área: Imagens Médicas
- Como estimar o volume de líquido amniótico em uma imagem fetal em apenas segundos?
- Temas: deep learning, processamento de imagens, quantificação de incerteza
- Resultado: algoritmo implementado como uma ferramenta de plugin – 92% de acurácia em menos de 5s, com intervalos preditivos.

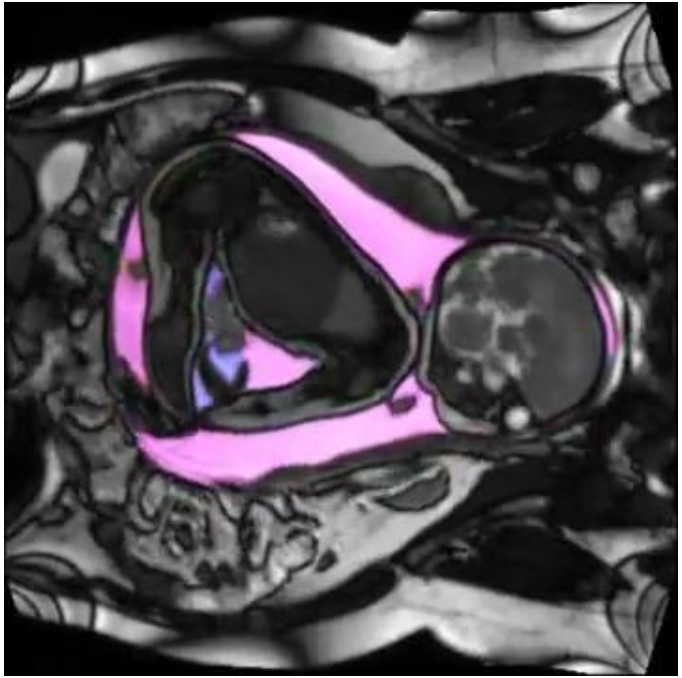
Projeto: DASA

TECNOLOGIA

Inteligência artificial criada por matemáticos brasileiros previne doenças na gravidez

Com uso de imagens de ressonância magnética, algoritmo teve taxa de 93% de acerto

12.set.2021 às 19h13



Fonte: https://www1.folha.uol.com.br/ciencia/2021/09/inteligencia-artificial-criada-por-matematicos-brasileiros-previne-doencas-na-gravidez.shtml

Projeto: Stone Pagamentos

- Área: Nota de Crédito
- Como calcular a probabilidade de um cliente dar calote?
- Temas: machine learning, processamento de dados, otimização, probabilidade
- Resultado: algoritmo superando soluções internas, paper publicado, ofertas de estágio aos alunos participantes

Projeto: Stone Pagamentos

impa
Instituto de Matemática Pura e Aplicada

Optimizing Combinatorial and Non-decomposable Metrics with ExactBoost

Daniel Csillag*, Carolina Piazza†, Thiago Ramos*, João Vitor Romano*, Roberto Oliveira*, Paulo Orenstein*

* Instituto de Matemática Pura e Aplicada, Rio de Janeiro, Brazil, † Princeton University, Princeton, USA

Summary

Combinatorial and non-decomposable losses pose computational and theoretical challenges, with many classification algorithms resorting to surrogates. In this work, we:

- introduce a fast and exact stagewise optimization algorithm, dubbed ExactBoost, that boosts stumps to the actual loss function;
- develop a novel extension of margin theory to the non-decomposable setting, recovering a guarantee previously available only to decomposable losses (Bartlett et al. 1998; Koltchinskii and Panchenko 2002; Schapire and Freund 2013);
- bound the generalization error of ExactBoost for metrics with different levels of non-decomposability, e.g. area under the ROC curve (AUC) and Kolmogorov-Smirnov (KS);
- empirically evaluate ExactBoost's performance against surrogate-based and exact algorithms available, showing it is generally superior, especially as an ensembler.

Overview of ExactBoost

The main idea behind ExactBoost is stagewise optimization tailored to a margin-adjusted empirical loss. Take $(X_1, y_1), \ldots, (X_n, y_n) \overset{\text{iid}}{\sim} \mathcal{D}$ and a loss $\widehat{L}$ that is invariant under rescaling and translation in its first argument. The goal is to find a score function where higher scores $S(X_i)$ should indicate higher likelihood of $y_i = 1$. It will be assumed that, after T rounds, a score has the form $S(X_i) = \sum_{t=1}^{T} w_t h_t(X_i)$, with $w_t \geq 0$, $\sum_{t=1}^{T} w_t = 1$, $h_t \in \mathcal{H}$, and the set of weak learners $\mathcal{H}$ are binary stumps of the form

$$\mathcal{H} = \left\{ \pm \mathbf{1}_{[X_{(j)} \leq \xi]} \pm \mathbf{1}_{[X_{(j)} > \xi]} \,:\, \xi \in \mathbb{R}, j \in [p] \right\}. \tag{1}$$

In order to attenuate overfitting, we consider the margin-adjusted loss $\widehat{L}_\theta(\mathbf{S}, \mathbf{y}) = \widehat{L}(\mathbf{S} - \theta\mathbf{y}, \mathbf{y})$, where $\theta > 0$ is a margin parameter.

The method works by picking (ξ_t, j_t, a_t, b_t) that solves

$$\min_{\xi \in \mathbb{R}, j \in [p], \tilde{a}, \tilde{b} \in \mathbb{R}} \widehat{L}\left(\mathbf{S}_{t-1} + \tilde{a}\mathbf{1}_{[X_{(j)} \leq \xi]} + \tilde{b}\mathbf{1}_{[X_{(j)} > \xi]} - (1 + |\tilde{b} - \tilde{a}|/2)\theta\mathbf{y}, \mathbf{y}\right), \tag{2}$$

and setting $\mathbf{S}_t = \mathbf{S}_{t-1} + a_t\mathbf{1}_{[X_{(j_t)} \leq \xi_t]} + b_t\mathbf{1}_{[X_{(j_t)} > \xi_t]}$.

The resulting algorithm is called ExactBoost, as it is based on the exact loss function provided rather than a surrogate. Note it takes as input an initial set of scores which could be any starting point, including scores trained by other learning models.

Algorithm 1: ExactBoost

function ExactBoost(data $(\mathbf{X}, \mathbf{y})$, initial scores S_0, margin θ), bins b, iterations T, estimator runs E)
for $e \in \{1, \ldots, E\}$ **do**
  $S_e \leftarrow S_0$
  for $t \in \{1, \ldots, T\}$ **do**
    $\mathbf{X}^0, \mathbf{y}^0 \leftarrow$ subsample $\mathbf{X}, \mathbf{y}$
    for $j \in \{1, \ldots, p\}$ **do**
      $\widehat{L}(h) \leftarrow \widehat{L}_\theta(S_e(\mathbf{X}^0_{[j]}) + h(\mathbf{X}^0_{[j]}), \mathbf{y}^0)$
      $h_j \leftarrow \operatorname{argmin}_h \widehat{L}(h)$ //Algorithm 2
    end for
    $h \leftarrow \operatorname{argmin}_{h_j} \widehat{L}_\theta(S_e(\mathbf{X}^0) + h_j(\mathbf{X}^0), \mathbf{y}^0)$
    $S_e^t \leftarrow S_e + h$
    if $\widehat{L}_\theta(S_e^t(\mathbf{X}), \mathbf{y}) \leq \widehat{L}_\theta(S_e(\mathbf{X}), \mathbf{y})$ **then**
      $S_e \leftarrow S_e^t$
      $S_e \leftarrow (S_e - \min S_e)/(\max S_e - \min S_e)$
    end if
  end for
end for
return $\operatorname{mean}(S_1, \ldots, S_E)$

Algorithm 2: Iterative Minimization

function Min(loss L_θ, data $\mathbf{X}_{[j]}$, labels $\mathbf{y}$, scores S, margin θ)
$\Xi \leftarrow \{\min \mathbf{X}_{[j]}, \max \mathbf{X}_{[j]}\}$
$A \leftarrow \{-1, 1\}$; $B \leftarrow \{-1, 1\}$
for $k \in \{1, \ldots, c\}$ **do**
  $l_k \leftarrow +\infty$
  for bisections $(\Xi^{(b)}, A^{(b)}, B^{(b)})$ **do**
    for $i \in \{1, \ldots, n\}$ **do**
      $s \leftarrow S + A^{(b)}\mathbf{1}_{[\mathbf{X}_{(j)} \leq \Xi^{(b)}]} + B^{(b)}\mathbf{1}_{[\mathbf{X}_{(j)} > \Xi^{(b)}]}$
      $\mathbf{s}_i \leftarrow S_j$ if $y_i = 0$ otherwise $\widetilde{S}_i$
    end for
    $l_k \leftarrow \widehat{L}_\theta(\mathbf{s}, \mathbf{y})$ if $\widehat{L}_\theta(\mathbf{s}, \mathbf{y}) < l_k$
  end for
end for
$l_\Xi \leftarrow \{\Xi, \Xi\}$; $I_\mathbf{A} \leftarrow \{\mathbf{A}, \mathbf{A}\}$; $I_\mathbf{B} \leftarrow \{\mathbf{B}, \mathbf{B}\}$
$S(a, b, \xi) \leftarrow S + a\mathbf{1}_{[\mathbf{X}_{(j)} \leq \xi]} + b\mathbf{1}_{[\mathbf{X}_{(j)} > \xi]}$
$(\xi_*, a_*, b_*) \leftarrow \operatorname{argmin}_{\xi \in l_\Xi, a \in I_\mathbf{A}, b \in I_\mathbf{B}} \widehat{L}_\theta(S(a, b, \xi), \mathbf{y})$
return $S + a_*\mathbf{1}_{[\mathbf{X}_{(j)} \leq \xi_*]} + b_*\mathbf{1}_{[\mathbf{X}_{(j)} > \xi_*]}$

Theoretical Results

Notation Let $\mathcal{D}$ be a probability distribution over $(X, y) \in \mathbb{R}^p \times \{0, 1\}$, and $\mathcal{D}_0$ (respectively, $\mathcal{D}_1$) denote the conditional distribution of X when $y = 0$ (respectively, 1). Conditionally on n_1 and n_0 respectively, the subsamples $\mathbf{X}_1 := (X_i \,:\, i \in [n], y_i = 1)$ and $\mathbf{X}_0 := (X_i \,:\, i \in [n], y_i = 0)$ are iid from $\mathcal{D}_1$ and $\mathcal{D}_0$. Score functions $S : \mathbb{R}^p \to [-1, 1]$ are convex combinations of elements in a family of measurable functions $\mathcal{H} : \mathbb{R}^p \to [-1, 1]$. Denote by $\mathcal{R}_n(\mathcal{H})$ the Rademacher complexity $\mathcal{H}$ with respect to $\mathcal{D}$ and $\mathcal{R}_{n,y}(\mathcal{H})$ for $y \in \{0, 1\}$ the complexities with respect to $\mathcal{D}_0$ and $\mathcal{D}_1$. Note $\mathcal{R}_n(\mathcal{H}) = O(\sqrt{\log p/n})$ when the set of weak learners $\mathcal{H}$ is as in (1).

Define the populational AUC and the populational KS losses as

$$\mathrm{AUC}(S) := 1 - \Pr\{S(X) > S(X')\},$$

$$\mathrm{KS}(S) := 1 - \sup_{t \in \mathbb{R}} \left(\Pr_{X \sim \mathcal{D}_1}\{S(X) \leq t\} - \Pr_{X \sim \mathcal{D}_0}\{S(X) \leq t\} \right).$$

Theorem 1. *Given $\theta > 0$, $\delta \in (0, 1)$, and a class of functions $\mathcal{H}$ from $\mathbb{R}^p$ to $[-1, 1]$, the following holds with probability at least $1 - \delta$: for all score functions $S : \mathbb{R}^p \to [-1, 1]$ obtained as convex combinations of the elements of $\mathcal{H}$,*

$$\mathrm{AUC}(S) \leq \widehat{\mathrm{AUC}}_\theta(S) + \frac{4}{\theta}\zeta_{\mathrm{AUC}}(\mathcal{H}) + \sqrt{\frac{2\log(1/\delta)}{\min\{n_0, n_1\}}},$$

where $\zeta_{\mathrm{AUC}}(\mathcal{H}) = \mathcal{R}_{\min\{n_0, n_1\}, 0}(\mathcal{H}) + \mathcal{R}_{\min\{n_0, n_1\}, 1}(\mathcal{H})$.

For ExactBoost, where $\mathcal{H}$ is given by (1), the theorem implies that the score S produced by the algorithm satisfies $\mathrm{AUC}(S) \leq \widehat{\mathrm{AUC}}_\theta(S) + o(1)$ with high probability whenever $\min\{n_0, n_1\} \gg \theta^{-2}\log p$. This result can be extended to similar pairwise losses.

Theorem 2. *Given $\theta > 0$, $\delta \in (0, 1)$, and a class of functions $\mathcal{H}$ from $\mathbb{R}^p$ to $[-1, 1]$, the following holds with probability at least $1 - \delta$: for all score functions $S : \mathbb{R}^p \to [-1, 1]$ obtained as convex combinations of the elements of $\mathcal{H}$,*

$$\mathrm{KS}(S) \leq \widehat{\mathrm{KS}}_\theta(S) + \frac{8}{\theta}\zeta_{\mathrm{KS}}(\mathcal{H}) + \sqrt{\frac{\log(2/\delta)}{2}}\left(\frac{1}{\sqrt{n_0}} + \frac{1}{\sqrt{n_1}}\right),$$

where $\zeta_{\mathrm{KS}}(\mathcal{H}) = \mathcal{R}_{n_0, 0}(\mathcal{H}) + \mathcal{R}_{n_1, 1}(\mathcal{H}) + n_0^{-1/2} + n_1^{-1/2}$.

In words, a score that achieves a small margin-adjusted KS loss should, with high probability, have good performance on the population. In the specific case of ExactBoost, the theorem above yields with probability greater than $1 - \delta$,

$$\mathrm{KS}(S) \leq \widehat{\mathrm{KS}}_\theta(S) + C\sqrt{\frac{\theta^{-2}(1 + \log p) + \log(2/\delta)}{\min\{n_0, n_1\}}},$$

with $C > 0$ a universal constant. Treating δ as fixed, good training performance on the margin-adjusted loss leads to good generalization when the number of positive and negative examples in the data are much larger than $\theta^{-2}\log p$.

Subsampling Subsampling can help ExactBoost avoid overfitting. The next proposition is helpful in controlling its impact in the optimization procedure for some losses.

Proposition 1. *Let $\widehat{L}$ be either the $\widehat{\mathrm{AUC}}$ or the $\widehat{\mathrm{KS}}$ loss. Consider a subset of indices $I = I_0 \cup I_1 \subset [n]$ chosen independently and uniformly at random with equal number of positive and negative cases, $|I_0| = |I_1| = k$. Let $h_\mathbf{I}$ be the optimal stump over the reduced sample $\{(X_j, y_j)\}_{j \in I}$ and score S and h_* the optimal stump over the entire sample $\{(X_i, y_i)\}_{i \in [n]}$. Then,*

$$\mathbb{E}[\widehat{L}(S + h_\mathbf{I})] \leq \widehat{L}(S + h_*) + \frac{e}{k},$$

where the expectation is over the choice of I.

Using random subsets of observations in ExactBoost leads to an expected error close to optimal when the subset has a balanced proportion of positive and negative examples.

Experiments

Benchmarks ExactBoost is compared to many learning algorithms: AdaBoost, k-nearest neighbors, logistic regression and random forest (via their Scikit-Learn implementation), gradient boosting (via XGBoost) and a 4-layer neural net (via TensorFlow). Methods that specifically optimize the performance metric are also considered: RankBoost (Freund et al. 2003) for AUC and DMKS (Fang and Chen 2019) for KS.

Hyperparameters Hyperparameters were fixed throughout the experiments. Baseline models were trained with the package-provided hyperparameters. Aided by experimental evidence on held-out datasets, ExactBoost uses $E = 250$ runs, $T = 50$ rounds, subsampling of 20% and margin of $\theta = 0.05$.

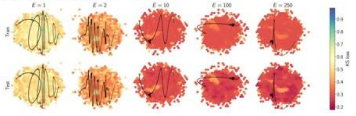


Fig 1: KS loss landscape highlighting ExactBoost's optimization trajectories on dataset heart via UMAP, going from left to right. More runs E lead to better train and test losses.

Datasets and timings Table 1 displays the main characteristics of each dataset, which span various applications, and range from balanced to imbalanced. Timing comparisons against exact benchmarks are also provided.

Dataset	Obs.	Features	Positives	RankBoost	DMKS
a1a	1605	119	24.6%	3.10x	34.50x
german	1000	20	70.0%	11.10x	2.00x
gisette	6000	5000	50.0%	OOT	36.70x
gmsc	150000	10	6.7%	OOT	87.50x
heart	303	21	45.9%	1.90x	23.70x
iono	351	34	64.1%	2.90x	4.70x
liver	145	5	37.9%	3.50x	17.40x

Table 1: Dataset properties and timings of algorithms relative to ExactBoost. ExactBoost is always faster (> 1×). OOT indicates the time budget of 5 days was exceeded (see the paper for details on computational setup).

Results Figure 2 shows that ExactBoost has good performance against surrogate alternatives. Results in the paper show that ExactBoost is also generally better as a standalone estimator than loss-specific optimizers such as RankBoost and DMKS.

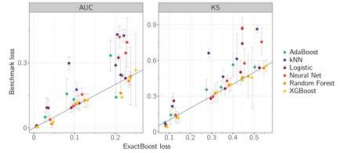


Fig 2: Test error for ExactBoost vs surrogate methods as estimators. Alternatives are generally worse than ExactBoost or statistically indistinguishable.

Ensembling ExactBoost is also a great ensembler. Six base models were used: AdaBoost, k-nearest neighbors, logistic regression, neural network, random forest and XGBoost. These models were trained on training folds, and their predictions on test folds were used as features for the ensemble models. Tables 2 and 3 show the results of using different surrogate and exact models as ensemblers.

Dataset	ExactBoost	AdaBoost	Logistic	XGBoost	RankBoost
a1a	$\mathbf{0.13 \pm 0.0}$	0.17 ± 0.0	0.14 ± 0.0	0.28 ± 0.1	0.16 ± 0.0
german	$\mathbf{0.23 \pm 0.0}$	0.32 ± 0.0	0.24 ± 0.0	0.35 ± 0.0	0.30 ± 0.1
gisette	$\mathbf{0.00 \pm 0.0}$	0.01 ± 0.0	0.01 ± 0.0	0.02 ± 0.0	0.01 ± 0.0
gmsc	0.15 ± 0.0	$\mathbf{0.14 \pm 0.0}$	0.31 ± 0.0	0.41 ± 0.0	0.15 ± 0.0
heart	$\mathbf{0.12 \pm 0.0}$	0.18 ± 0.1	$\mathbf{0.12 \pm 0.0}$	0.23 ± 0.1	0.15 ± 0.0
iono	$\mathbf{0.04 \pm 0.0}$	0.05 ± 0.0	0.07 ± 0.0	0.09 ± 0.0	0.05 ± 0.0
liver	$\mathbf{0.30 \pm 0.1}$	0.34 ± 0.1	0.34 ± 0.1	0.38 ± 0.0	0.38 ± 0.1

Table 2: Evaluation of AUC ensemblers. ExactBoost is generally the top performer.

Dataset	ExactBoost	AdaBoost	Logistic	XGBoost	DMKS
a1a	$\mathbf{0.37 \pm 0.1}$	0.44 ± 0.1	0.40 ± 0.1	0.57 ± 0.1	0.49 ± 0.1
german	$\mathbf{0.50 \pm 0.1}$	0.68 ± 0.1	0.53 ± 0.1	0.69 ± 0.1	0.53 ± 0.1
gisette	$\mathbf{0.04 \pm 0.0}$	$\mathbf{0.04 \pm 0.0}$	0.07 ± 0.0	$\mathbf{0.04 \pm 0.0}$	0.10 ± 0.0
gmsc	$\mathbf{0.43 \pm 0.0}$	0.44 ± 0.0	0.73 ± 0.0	0.83 ± 0.0	0.46 ± 0.0
heart	$\mathbf{0.34 \pm 0.1}$	0.38 ± 0.1	0.37 ± 0.1	0.46 ± 0.1	0.40 ± 0.0
iono	$\mathbf{0.13 \pm 0.1}$	0.18 ± 0.1	0.18 ± 0.1	0.19 ± 0.1	0.27 ± 0.1
liver	$\mathbf{0.53 \pm 0.1}$	0.60 ± 0.2	0.59 ± 0.2	0.76 ± 0.0	0.60 ± 0.2

Table 3: Evaluation of KS ensemblers. ExactBoost is always the top performer.

Takeaways

The main takeaways from the proposed boosting algorithm are:

- first-order surrogate methods are widely used in classification tasks, but there is value to be gained in working with the intended loss function;
- ExactBoost's theoretical guarantees translate to compelling empirical performance;
- computational implementations can be made reasonably fast through interval arithmetic;
- we expect the margin extension and theoretical results presented to hold over more general classes of combinatorial and non-decomposable losses beyond AUC and KS (see the main paper for the precision at k loss).

Main References

Bartlett, Peter et al. (1998). "Boosting the margin: a new explanation for the effectiveness of voting methods". In: *Annals of Statistics* 26.5, pp. 1651–1686.

Fang, Fang and Yuanyuan Chen (2019). "A new approach for credit scoring by directly maximizing the Kolmogorov–Smirnov statistic". In: *Computational Statistics & Data Analysis* 133, pp. 180–194. ISSN: 0167-9473.

Freund, Yoav et al. (Dec. 2003). "An Efficient Boosting Algorithm for Combining Preferences". In: *J. Mach. Learn. Res.* 4, pp. 933–969. ISSN: 1532-4435.

Koltchinskii, V. and D. Panchenko (Feb. 2002). "Empirical Margin Distributions and Bounding the Generalization Error of Combined Classifiers". In: *Annals of Statistics* 30.1, pp. 1–50. DOI: 10.1214/aos/1015362183.

Schapire, Robert E and Yoav Freund (2013). "Boosting: Foundations and algorithms". In: *Kybernetes*. DOI: 10.1108/03684921311295547.

Projeto: Tribunal Superior Eleitoral

- Área: Modelagem de Eleição
- Adicionar uma hora extra de votação diminuiria a propagação da Covid-19? Deveria haver um horário específico para 60+?
- Temas: modelagem probabilística, teoria das filas, estatística descritiva
- Resultado: Relatório defendendo a implementação de uma hora extra

Projeto: Tribunal Superior Eleitoral

POLÍTICA • BRASÍLIA

Coronavírus: votação nos dias das eleições deve ser estendida em pelo menos uma hora, diz Barroso

Para evitar aglomerações, TSE também avalia reservar horários para idosos

Washington Luiz
05/08/2020 - 13:07 / Atualizado em 05/08/2020 - 13:39

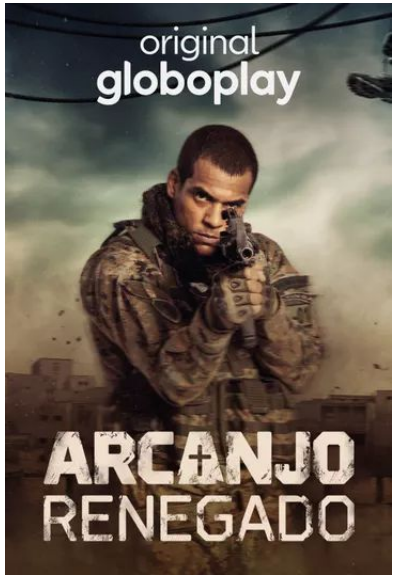
O presidente do TSE, ministro Luís Roberto Barroso, apresenta dados do eleitorado brasileiro Foto: Reprodução/ Rede Social

Fonte:
https://oglobo.globo.com/politica/coronavirus-votacao-nos-dias-das-eleicoes-deve-ser-estendida-em-pelo-menos-uma-hora-diz-barroso-1-24568930

Projeto: Cartesi

- Área: blockchains, contratos inteligentes
- A “infraestrutura de blockchain” da Cartesi está funcionando de forma apropriada? Como monitorar erros, fraudes e mal comportamento ao mesmo tempo?
- Temas: métodos numéricos, estatística online e machine learning
- Resultado: algoritmos detectando erros numéricos, detecção de anomalias com garantias de corretude matemática

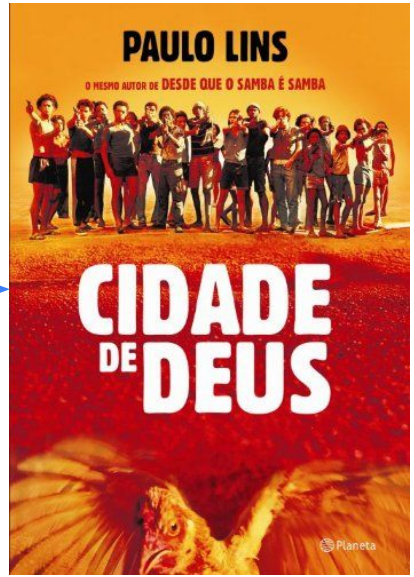
Metadados



- Título: Arcanjo Renegado
- Elenco: Marcelo Mello Jr., Erika Januza
- Sinopse: A série conta a história de Mikhael (Marcelo Mello Jr.), líder da principal equipe do BOPE. Quando um dos seus amigos é ferido em uma operação, ele busca vingança e acaba em conflito com a alta cúpula política do estado.

Recomendar a Partir de Metadados

?



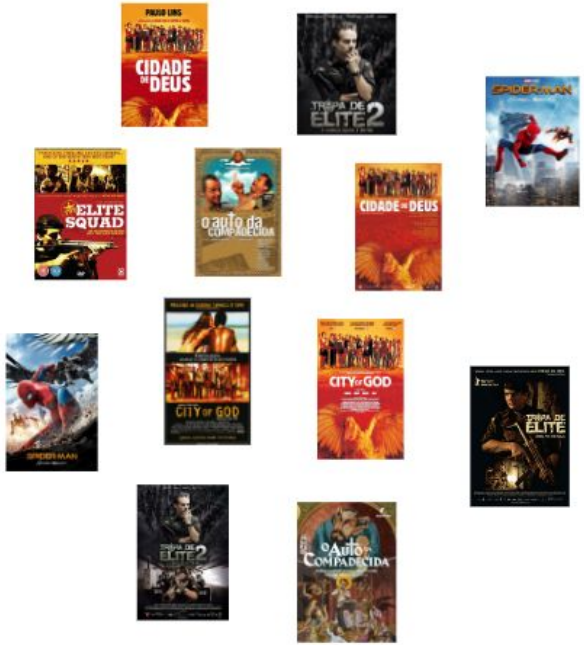
Como desenhamos um sistema de recomendação baseado nos metadados?

anos

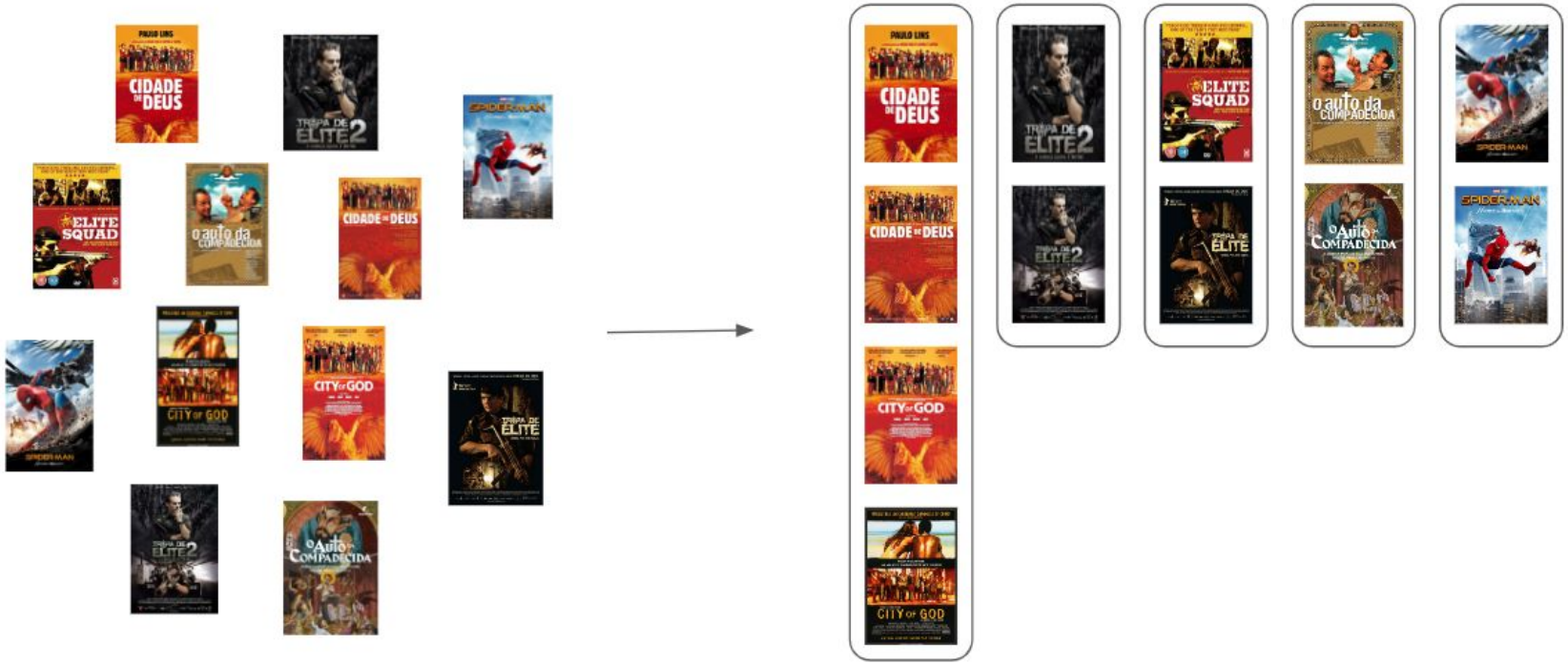
impa

Instituto de Matemática
Pura e Aplicada

Passo 1: Agregação



Passo 1: Agregação



Passo 2: Extração de Metadados

A série conta a história de Mikhael (Marcelo Mello Jr.), líder da principal equipe do BOPE. Quando um dos seus amigos é ferido em uma operação, ele busca vingança e acaba em conflito com a alta cúpula política do estado.

Passo 2: Extração de Metadados

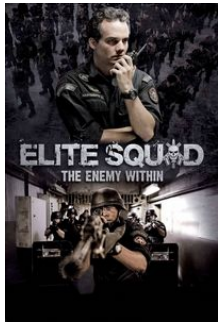
Entidade

A série conta a história de Mikhael (Marcelo Mello Jr.), líder da principal equipe do BOPE. Quando um dos seus amigos é ferido em uma operação, ele busca vingança e acaba em conflito com a alta cúpula política do estado.

Corrupção

Tema

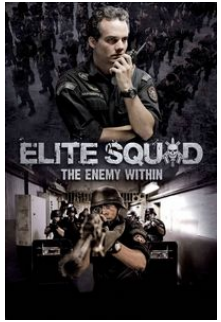
Step 3: Recomendação

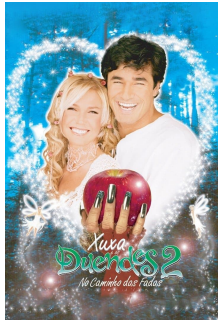
$similaridade = \rho($  $,$ $)$

Step 3: Recomendação

$\rho($

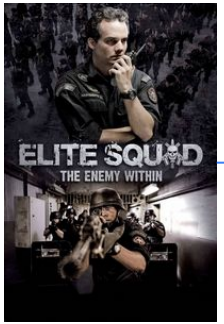
,

$) > \rho($ 

, 

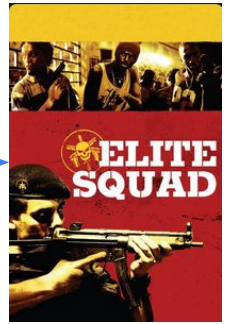
$)$

Step 3: Recomendação



$$\begin{matrix} x_1 \\ \vdots \\ x_n \end{matrix} \qquad \rho(\vec{x}, \vec{y}) \qquad \begin{matrix} y_1 \\ \vdots \\ y_n \end{matrix}$$

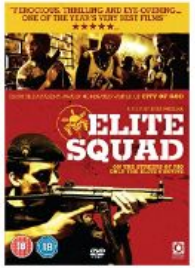
Step 3: Recomendação

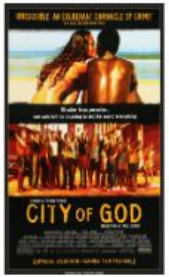


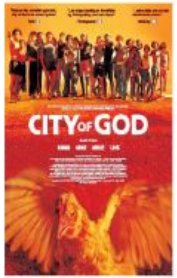
Uma nova ferramenta para resolução de identidades

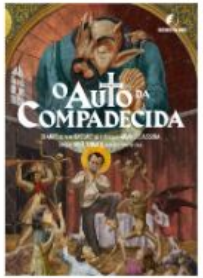
impa
Instituto de Matemática
Pura e Aplicada

Resolução de Identidade

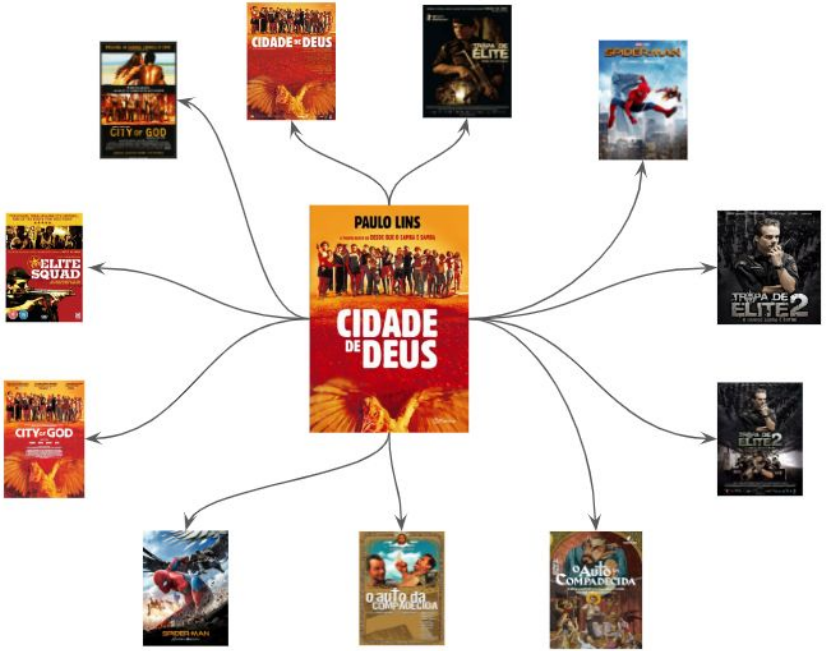






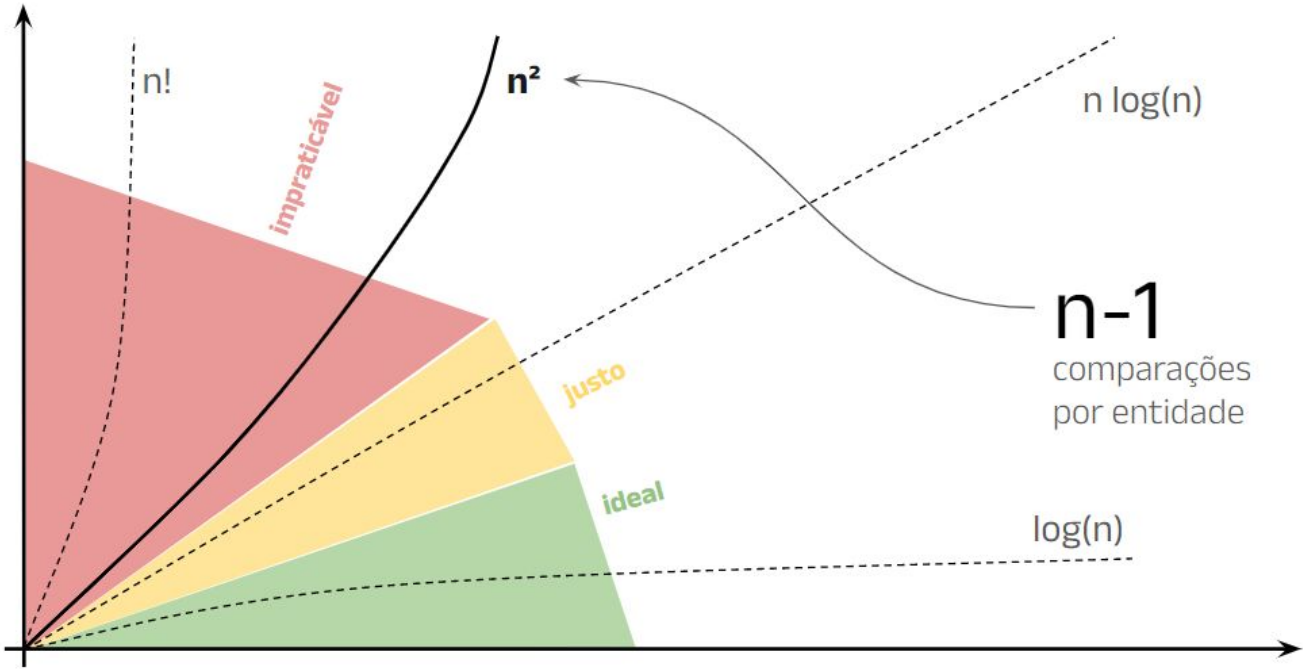


Resolução de Identidade

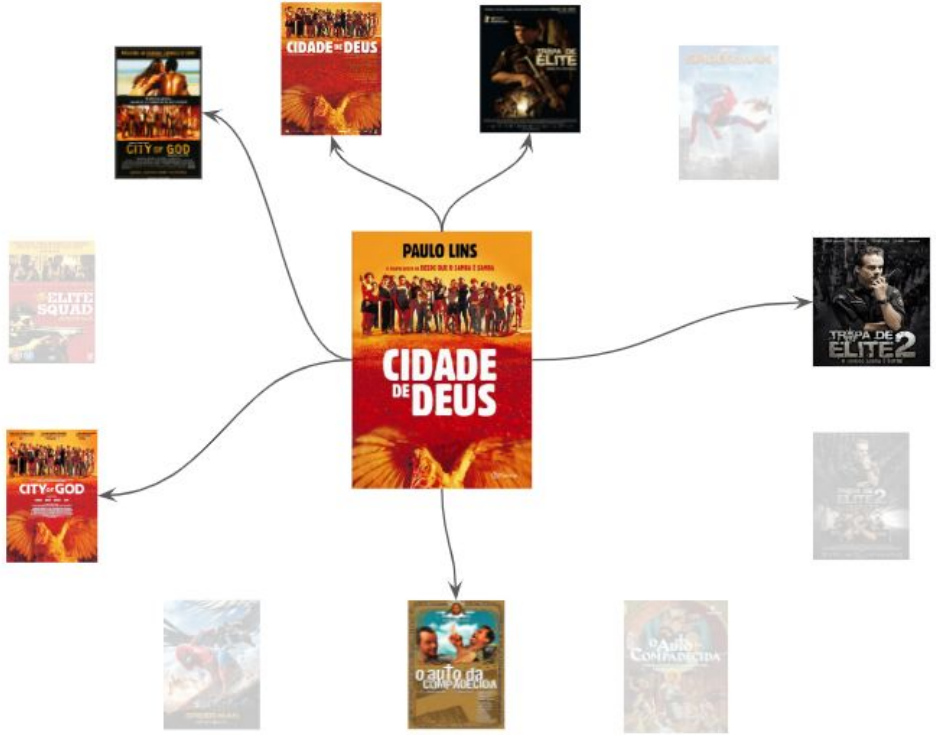


n-1
comparações
por entidade

Computacionalmente Inviável



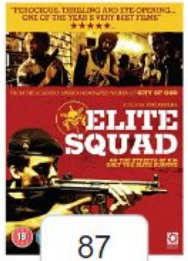
Resolução de Identidade via Blocos



$\ll n$

comparações por entidade

Criando Blocos via Hashing



87

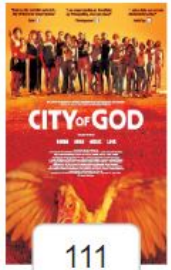
110

9

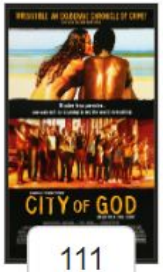
110

87

9



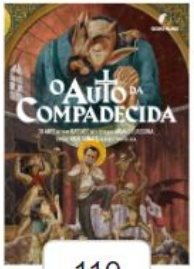
111



111

110

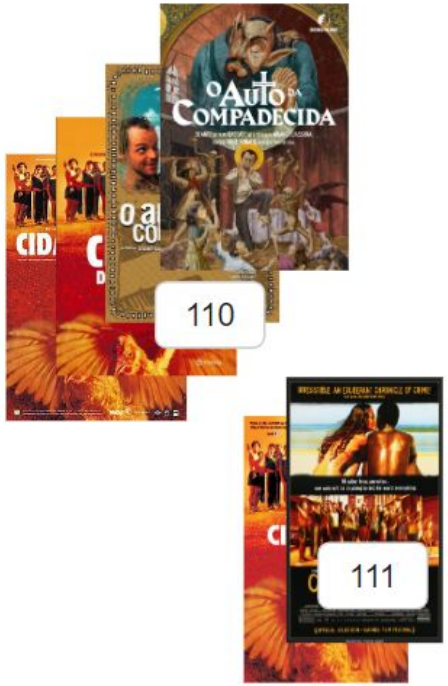
88



110

87

Objetivo: Induzir Poucos Testes

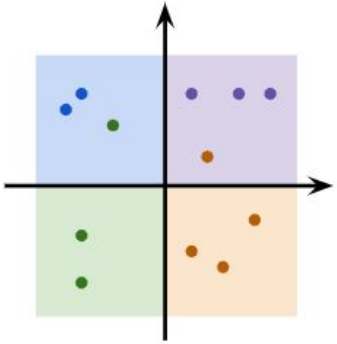


Aprender o Hash Para Cada Contexto

hashes fixos

Abundantes na literatura

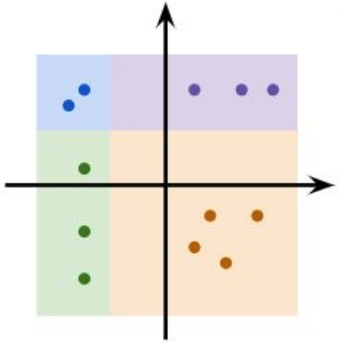
Não necessitam de treino

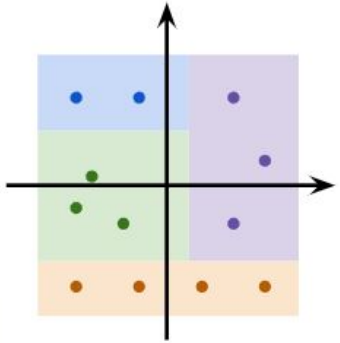


hashes aprendidos

Métodos recentes e pouco explorados

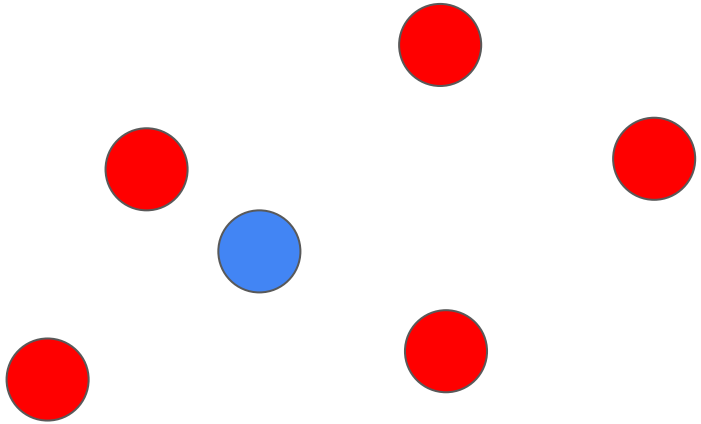
Permitem aprender a geometria do espaço de features





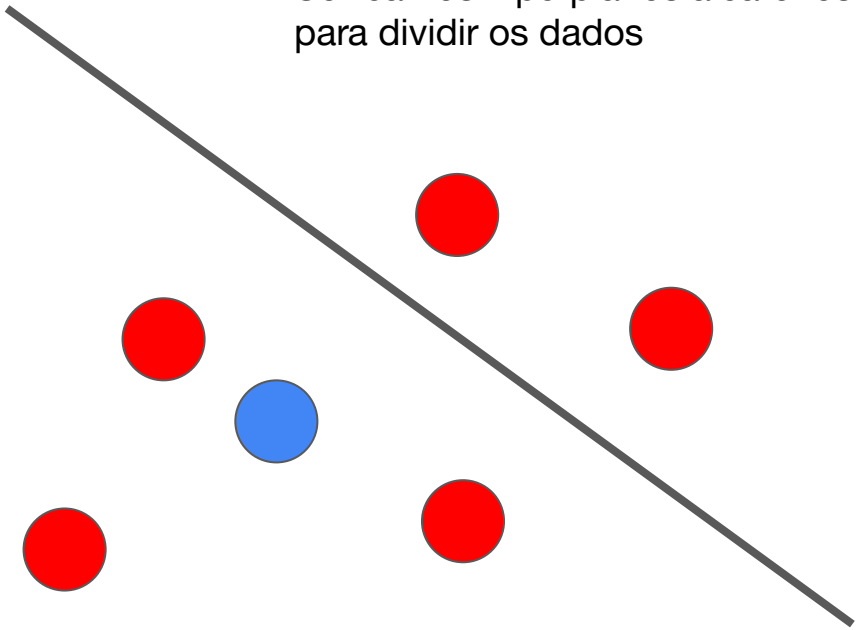
Exemplo de Hash Fixo: LSH

Como encontrar o elemento mais próximo
da bolinha azul, sem fazer todas as comparações
possíveis?



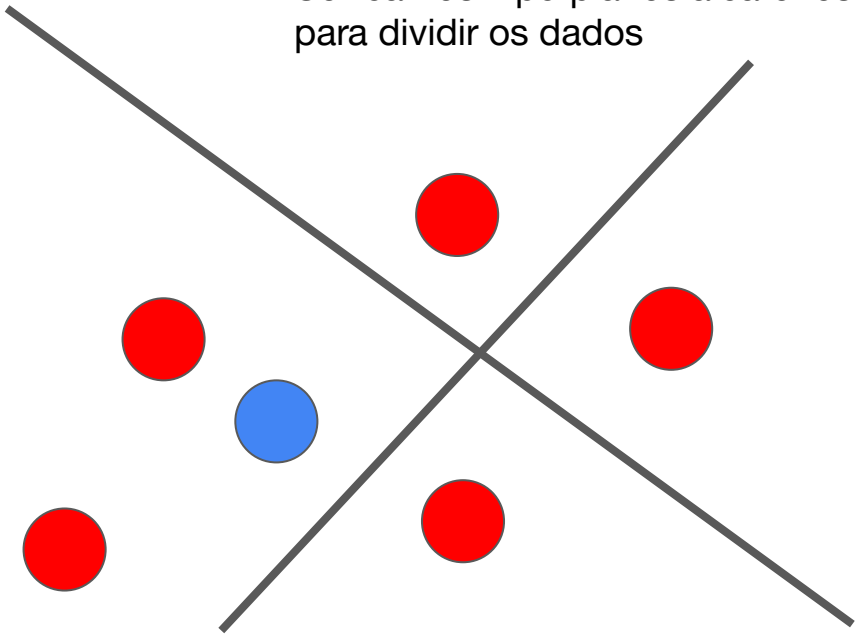
Exemplo de Hash Fixo: LSH

Sorteamos hiperplanos aleatórios
para dividir os dados

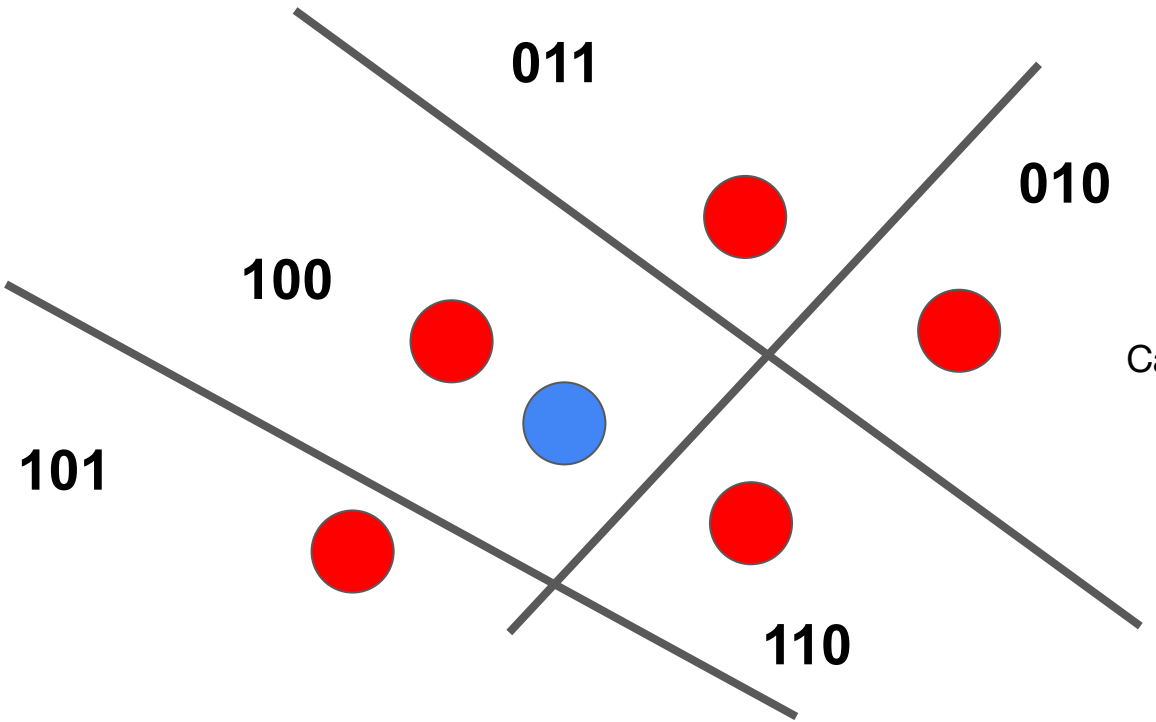


Exemplo de Hash Fixo: LSH

Sorteamos hiperplanos aleatórios
para dividir os dados



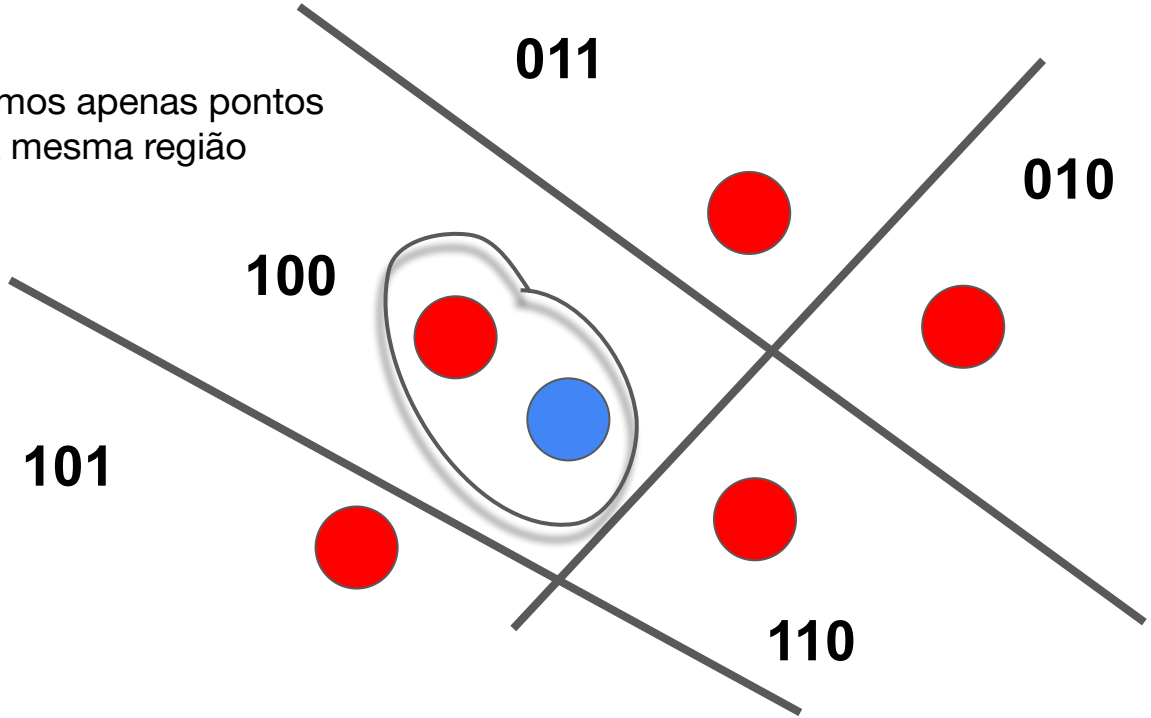
Exemplo de Hash Fixo: LSH



Cada região final define um hash

Exemplo de Hash Fixo: LSH

Comparamos apenas pontos dentro da mesma região



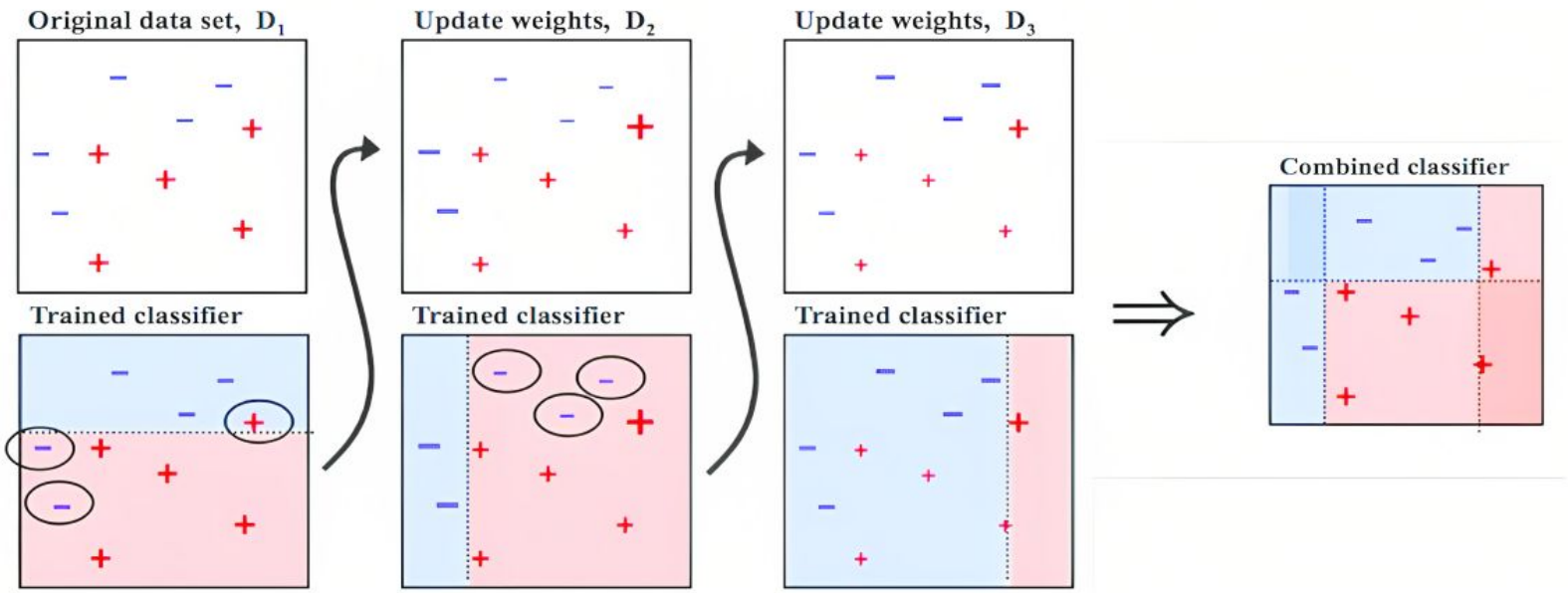
Exemplo de Hash Fixo: LSH

- Com alta probabilidade, esse método encontra o elemento mais próximo
- Existem extensões desse método para que os hashes criados sejam aprendidos através dos dados:
  - Kernelized Locality-Sensitive Hashing (KLSH)
  - Transitive Locality-Sensitive Hashing (TLSH)
- Porém esses métodos são pouco escaláveis e/ou não possuem garantias teóricas

Como adaptar LSH?

Conseguimos adaptar o método de LSH usando algum algoritmo de Machine Learning escalável e com garantias teóricas?

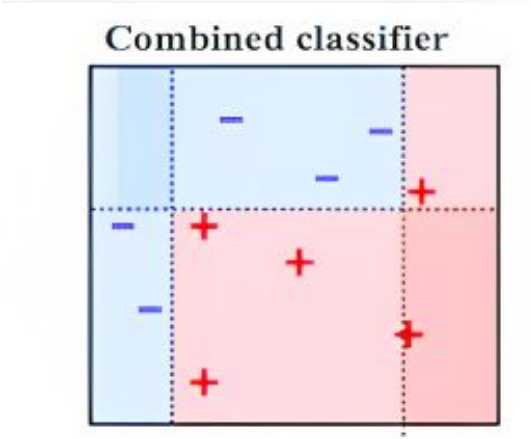
Boosting



Fonte: https://towardsdatascience.com/understanding-adaboost-for-decision-tree-ff8f07d2851

Boosting

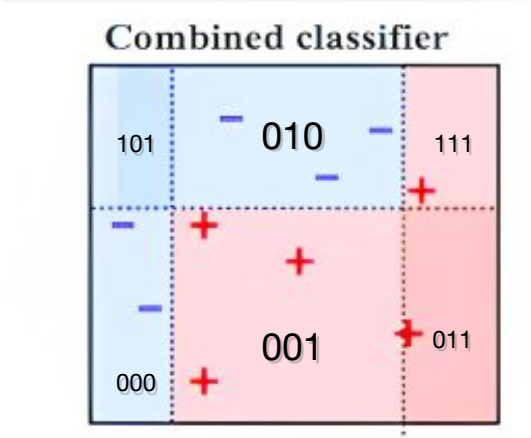
Boosting nos dá uma forma natural para criação de hashes!



Fonte: https://towardsdatascience.com/understanding-adaboost-for-decision-tree-ff8f07d2851

Boosting

Boosting nos dá uma forma natural para criação de hashes!



Fonte: https://towardsdatascience.com/understanding-adaboost-for-decision-tree-ff8f07d2851

BlockBoost: Combinando Boosting e LSH

Produzimos um algoritmo, *BlockBoost*, que funciona em duas etapas:

1. **(Boosting)** Aprendemos <u>regiões de separação</u> dos pontos usando Boosting
    a. Extensa teoria
    b. Algoritmos otimizados
2. **(Hashing)** Através dessas regiões e utilizando técnicas de LSH, criamos hashes que maximizam a probabilidade de pontos semelhantes serem comparados
    a. Escalável
    b. Garantias teóricas

BlockBoost: Etapa de Boosting

Algorithm 1 Boosting step

Input: $\mathcal{S}_{\text{train},n} = ((a_i, b_i), y_i)_{i=1}^n$, number of iterations $T \in \mathbb{N}$, binary family $\mathcal{K}$

for $i \leftarrow 1$ to n **do**
  $Q_1(i) \leftarrow \frac{1}{n}$
end for
for $t \leftarrow 1$ to T **do**
  $k_t^* \leftarrow$ classifier in $\mathcal{K}$ with error $\varepsilon_t = \sum_{i=1}^n Q_t(i) \mathbf{1}_{[y_i k_t^*(a_i) k_t^*(b_i) < 0]} < 1/2$
  $\alpha_t' \leftarrow \frac{1}{2} \log\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right) > 0$
  $Z_t \leftarrow 2\left[\varepsilon_t(1-\varepsilon_t)\right]^{1/2}$
  for $i \leftarrow 1$ to n **do**
    $Q_{t+1}(i) \leftarrow \frac{Q_t(i) \exp\left(-\alpha_t' y_i k_t^*(a_i) k_t^*(b_i)\right)}{Z_t}$
  end for
end for
for $t \leftarrow 1$ to T **do**
  $\alpha_t^* \leftarrow \frac{\alpha_t'}{\sum_{s=1}^T \alpha_s'}$
end for

Output: $(\alpha_t^*)_{t=1}^T, (k_t^*)_{t=1}^T$

BlockBoost: Etapa de Hashing

Algorithm 2 Hashing step

Input: $k, L \in \mathbb{N}$, convex weights $(\alpha_t^*)_{t=1}^T$, classifiers $(k_t^*)_{t=1}^T$

```
for i ← 1 to L do
    for j ← 1 to k do
        g_{i,j} ← k_t^* with probability α_t^*
    end for
    g_i ← (g_{i,1}, ..., g_{i,k})
end for
```

Output: $(g_1, \ldots, g_L)$

BlockBoost: Probabilidade de Colisão

Lemma A.1. *Let f^* be as in (2). Then for any $(A, B) \in \mathcal{A} \times \mathcal{B}$, and any function $g_{i,j}$ as in Algorithm 2,*

$$\mathbb{P}_{g_{i,j}}\left[g_{i,j}(A) = g_{i,j}(B)\right] = \frac{1 + f^*(A, B)}{2},$$

where the probability is over the choice of $g_{i,j}$.

Isto é, se dois elementos são a mesma entidade, a probabilidade de um bit do nosso hash coincidir é maior que 50%. E é menor que 50% caso contrário.

BlockBoost: Resultados Teóricos

Theorem 4.3 (Performance of the hashing step). *Consider databases $\mathcal{A}$ and $\mathcal{B}$ such that $|\mathcal{A}| = N_{\mathcal{A}}$ and $|\mathcal{B}| = N_{\mathcal{B}}$. For given $\theta > 0$ and $\gamma \in (0,1)$, if the output f^* of Algorithm 1 satisfies the θ-margin condition and we set:*

$$\rho := \frac{\log\left(\frac{2}{1+\theta}\right)}{\log\left(\frac{2}{1-\theta}\right)} \in [0,1),$$

$$k := \lceil \log_{\frac{2}{1+\theta}} N_{\mathcal{A}} \cdot N_{\mathcal{B}} \rceil,$$

$$L := \left\lceil \frac{2(N_{\mathcal{A}} \cdot N_{\mathcal{B}})^{\rho} \log(1/\gamma)}{1+\theta} \right\rceil,$$

then Algorithm 2 achieves

$$\mathbb{E}\left[\text{Recall}\right] \geq (1-\gamma)(1-\varepsilon)$$

$$\mathbb{E}\left[\text{RR}\right] \geq \left(1 - \frac{|\mathcal{M}| + L}{N_{\mathcal{A}} \cdot N_{\mathcal{B}}}\right)(1-\varepsilon),$$

where expectations are with respect to the randomness in the hash code, and Recall and RR are defined in (4) and (5).

BlockBoost: Resultados Empíricos

Table 1: Baseline datasets

#	DATASET	ENTITIES	MATCHES	TABLES	FEAT.
1	ABT_BUY	2,173	1,097	2	4
2	AMZ_GG	4,589	1,300	2	5
3	DBLP_ACM	4,908	2,224	2	4
4	DBLP_SCH	66,879	5,347	2	4
5	RESTAURANT	865	752	1	4
6	RLDATA10K	10,000	1,000	1	8
7	RLDATA500	500	50	1	8
8	WM_AMZ	24,583	1,145	2	29

BlockBoost: Resultados Empíricos

Table 2: Comparison table of the evaluation metric H(Recall, RR), defined in 7, with the two best models per dataset **highlighted**. Note that even when BlockBoost is not highlighted, its performance is close to the best algorithm.

#	BlockBoost	Canopy	LSH	KLSH	TLSH	Attribute	Quadratic
1	**0.736**	**0.800**	0.570	0.418	0.644	0.000	0.085
2	**0.628**	**0.651**	0.558	0.626	0.541	0.014	0.420
3	**0.971**	0.878	0.760	**0.979**	0.860	0.462	0.961
4	**0.966**	0.870	0.751	**0.962**	0.862	0.419	0.847
5	**0.961**	0.846	0.925	**0.997**	0.905	0.542	0.932
6	0.990	0.948	0.574	0.968	**0.993**	**0.993**	0.990
7	0.998	0.884	0.987	0.995	**0.999**	**0.999**	**0.999**
8	**0.725**	**0.707**	0.582	0.617	0.465	0.683	0.540
AVG	**0.872**	**0.823**	0.713	0.820	0.784	0.514	0.722

anos

impa

Instituto de
Matemática
Pura e Aplicada

Estrada Dona Castorina, 110 Jardim Botânico
22460-320, Rio de Janeiro, RJ – Brasil
(21) 2529-5000 | impa.br